AI AGENT MASTERY

Week 2: Framework Basics

LangGraph and ADK Focus

16 Core Topics | 3 Hands-On Exercises | 1 Ongoing Research Project

Estimated Time: 12-15 hours

Repository: https://github.com/kikiaitraining/ai-agent-mastery/tree/master/week-02-framework-basics

Learning Objectives

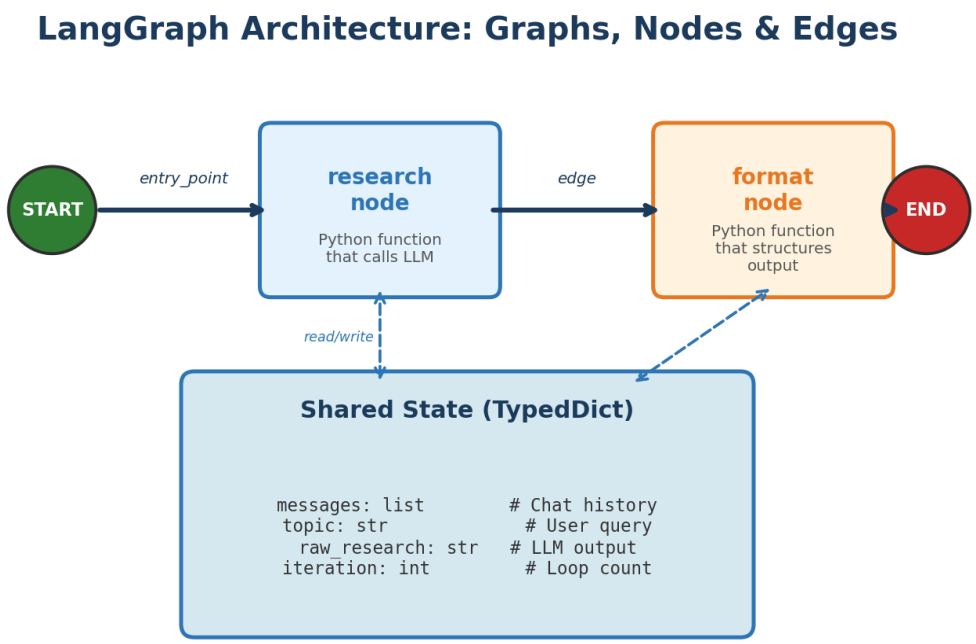
By the end of Week 2, you will be able to:

- Understand graph-based agent architecture (nodes, edges, state) in LangGraph
- Build tool-using agents that follow the ReAct pattern with LangGraph
- Manage complex state with TypedDict, reducers, and inter-node communication
- Use Google ADK's declarative approach: LlmAgent, Runner, and SessionService
- Build multi-agent workflows with SequentialAgent and AgentTool delegation
- Configure ADK agents via YAML for rapid iteration
- Compare LangGraph vs ADK trade-offs and choose the right framework for each task
- Implement retry logic with exponential backoff for resilient agents
- Track token usage and API costs with CostTracker utilities
- Add structured logging middleware to LangGraph nodes and ADK events
- Trace agent flows visually with Arize Phoenix
- Port an agent between frameworks to understand architectural differences

Topic 1: LangGraph Basics — Graphs, Nodes, Edges

Time: 1 hour

LangGraph is the core framework from the LangChain ecosystem for building agent workflows as directed graphs. Before using any LLM, it is essential to understand the graph abstraction itself: StateGraph, nodes (Python functions), edges (connections), and state (a shared TypedDict). This topic uses no LLM calls — just pure Python — so you can focus on how data flows through a graph.



Core Concepts

- **StateGraph:** The main class that holds your graph definition. You add nodes and edges to it, then compile() to create a runnable application.
- **Nodes:** Python functions that read from and write to shared state. Each node takes the state dict as input and returns a dict of fields to update.
- **Edges:** Connections between nodes. Linear edges always go one direction; conditional edges use a routing function to choose the next node.
- **State:** A TypedDict shared by all nodes. Every node can read any field and update specific fields. Nodes only return the fields they change.

Linear Graph: Nodes Run One After Another

The simplest graph pattern chains nodes sequentially: intake -> process -> output -> END. Here is the complete state definition and node functions from the repository:

example_01_langgraph_graph_basics.py — State and Node Definitions

```
# From example_01_langgraph_graph_basics.py
from langgraph.graph import StateGraph, END
from typing import TypedDict

class LinearState(TypedDict):
    input_text: str          # Plain field - each write REPLACES the old value
    processed_text: str      # Stores result after processing
    step_log: list           # Tracks which nodes ran

def intake_node(state: LinearState) -> dict:
    """First node: receives input and logs that we started."""
    return {"step_log": state.get("step_log", []) + ["intake"]}

def process_node(state: LinearState) -> dict:
    """Second node: transforms the input text."""
    text = state["input_text"]
    processed = f"{text.upper()} (word count: {len(text.split())})"
    return {"processed_text": processed, "step_log": state.get("step_log", []) + ["process"]}

def output_node(state: LinearState) -> dict:
    """Third node: finalizes the result."""
    return {"step_log": state.get("step_log", []) + ["output"]}
```

Building and running the graph:

Building the Linear Graph

```
def build_linear_graph():
    graph = StateGraph(LinearState)
    graph.add_node("intake", intake_node)
    graph.add_node("process", process_node)
    graph.add_node("output", output_node)
    graph.set_entry_point("intake")
    graph.add_edge("intake", "process")
    graph.add_edge("process", "output")
    graph.add_edge("output", END)
    return graph.compile()

app = build_linear_graph()
result = app.invoke({"input_text": "LangGraph makes agent workflows visual", "processed_text": "",
"step_log": []})
# result["step_log"] -> ["intake", "process", "output"]
```

Conditional Graph: Edges That Choose Where to Go

Conditional edges use a routing function that returns a string matching one of the edge targets. This enables loops — a node can route back to itself until a condition is met:

Conditional Graph with Loop

```
class ConditionalState(TypedDict):
    value: int
```

```
    step_count: int
    max_steps: int
    history: list

def should_continue(state: ConditionalState) -> str:
    """Decide: keep doubling or stop."""
    if state["step_count"] < state["max_steps"]:
        return "double"   # Loop back
    else:
        return "done"     # Move forward

graph = StateGraph(ConditionalState)
graph.add_node("init", init_node)
graph.add_node("double", double_node)
graph.add_node("done", done_node)
graph.set_entry_point("init")
graph.add_edge("init", "double")
graph.add_conditional_edges("double", should_continue, {"double": "double", "done": "done"})
graph.add_edge("done", END)
```

Branching Graph: Different Paths Based on Input

Branching graphs use conditional edges to route to different handler nodes. The routing function returns which branch to take, and each branch converges back to END:

Branching based on classification

```
def route_by_category(state: BranchState) -> str:
    return state["category"]  # Returns "short", "medium", or "long"

graph.add_conditional_edges("classify", route_by_category, {
    "short": "short", "medium": "medium", "long": "long"
})
graph.add_edge("short", END)
graph.add_edge("medium", END)
graph.add_edge("long", END)
```

Understanding State Reducers (add_messages)

Without a reducer, each node write REPLACES the entire field. The add_messages reducer changes this behavior to APPEND instead — this is how LangGraph handles conversation history:

Replace vs Append behavior

```
from typing import Annotated
from langgraph.graph import add_messages

class ReducerState(TypedDict):
    plain_list: list                              # No reducer: REPLACES on write
    reducer_list: Annotated[list, add_messages]    # add_messages: APPENDS on write

# Node A writes ["a","b"] -> Node B writes ["c","d"]
```

```
# plain_list final value:   ["c","d"]        (replaced)
# reducer_list final value: [msg1, msg2]     (both kept)
```

Key Insight

The add_messages reducer is critical for agent conversations. Without it, each agent node would overwrite the entire message history. Always use Annotated[list, add_messages] for your messages field.

Full source: example_01_langgraph_graph_basics.py

Topic 2: LangGraph — Simple Agent Creation

Time: 1 hour

Now we add an LLM to the graph. This builds a complete tool-using agent that follows the ReAct (Reason + Act) pattern: the LLM receives a question, decides to call a tool or respond directly, and loops until done. This is THE core pattern for LangGraph agents.

Step 1: Define Tools with @tool Decorator

LangGraph tools use the @tool decorator from langchain_core.tools. The decorator creates a schema from the function signature and docstring that the LLM reads to decide when to call each tool:

example_02_langgraph_tool_agent.py — Tool Definitions

```
from langchain_core.tools import tool

@tool
def calculate(expression: str) -> str:
    """Evaluate a math expression and return the result.
    Use this for any arithmetic: addition, subtraction, multiplication,
    division, exponents (use **), etc.
    Example inputs: '15 * 7', '2 ** 10', '100 / 3'
    """
    try:
        allowed_chars = set("0123456789+-*/.() ")
        if not all(c in allowed_chars for c in expression):
            return "Error: Invalid characters in expression."
        result = eval(expression)
        return f"{expression} = {result}"
    except ZeroDivisionError:
        return f"Error: Division by zero in '{expression}'"
    except Exception as e:
        return f"Error evaluating '{expression}': {e}"

@tool
def string_reverse(text: str) -> str:
    """Reverse a string. Use this when asked to reverse text or words."""
    return text[::-1]

@tool
def word_count(text: str) -> str:
    """Count the number of words in a text."""
    words = len(text.split())
    chars = len(text)
    return f"'{text}' has {words} words and {chars} characters"
```

Step 2: Bind Tools to the LLM

bind_tools() tells the LLM about available tools. The LLM will see each tool's name, description, and parameter schema:

LLM Setup and Tool Binding

```
tools = [calculate, string_reverse, word_count]

provider = os.getenv("LLM_PROVIDER", "groq").lower()
if provider == "groq":
    from langchain_groq import ChatGroq
    llm = ChatGroq(model=os.getenv("GROQ_MODEL", "llama-3.3-70b-versatile"))
else:
    from langchain_openai import ChatOpenAI
    llm = ChatOpenAI(model=os.getenv("OPENAI_MODEL", "gpt-4o-mini"))

llm_with_tools = llm.bind_tools(tools)  # LLM now knows about our tools
```

Step 3: State, Agent Node, and Routing

Agent Node and Routing Function

```
class AgentState(TypedDict):
    messages: Annotated[list, add_messages]  # Conversation history (appends)
    tool_call_count: int                      # Safety counter

def agent_node(state: AgentState) -> dict:
    """Call the LLM. It will either request a tool call or give a final answer."""
    for attempt in range(3):  # Retry for malformed tool calls
        try:
            response = llm_with_tools.invoke(state["messages"])
            return {"messages": [response]}
        except Exception as e:
            if attempt < 2:
                continue
            raise

MAX_TOOL_CALLS = 5  # Safety limit

def should_continue(state: AgentState) -> str:
    """Route: tools or end."""
    last_message = state["messages"][-1]
    if state.get("tool_call_count", 0) >= MAX_TOOL_CALLS:
        return "end"   # Safety valve
    if hasattr(last_message, "tool_calls") and last_message.tool_calls:
        return "tools"
    return "end"
```

Step 4: Build the Complete Graph

The full agent graph creates a loop: agent -> tools -> agent, continuing until the LLM gives a final answer (no tool calls):

Complete Agent Graph

```
graph = StateGraph(AgentState)
graph.add_node("agent", agent_node)
graph.add_node("tools", tools_node)        # ToolNode executes tool calls
graph.set_entry_point("agent")
graph.add_conditional_edges("agent", should_continue, {"tools": "tools", "end": END})
graph.add_edge("tools", "agent")           # After tools, always go back to agent
```

```
app = graph.compile()

# Run it:
result = app.invoke({
    "messages": [HumanMessage(content="What is 15 * 7?")],
    "tool_call_count": 0,
})
```

Key Insight

The agent -> tools -> agent loop IS the ReAct pattern. The LLM reasons about which tool to call, acts by calling it, observes the result, and repeats until it has the answer. Always add a safety valve (max tool calls) to prevent infinite loops.

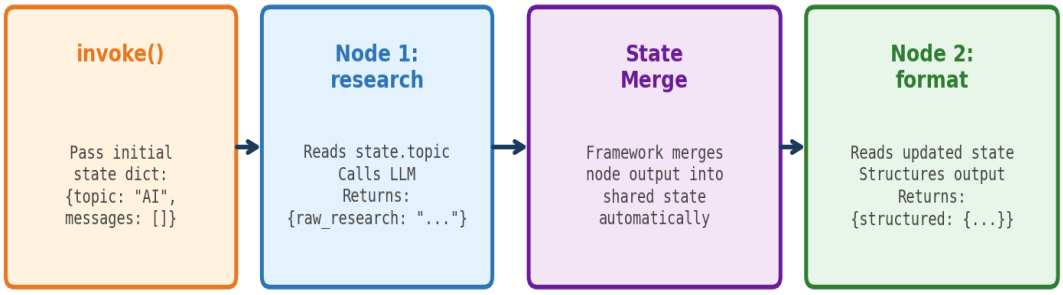
Full source: example_02_langgraph_tool_agent.py

Topic 3: LangGraph — State Management

Time: 45 minutes

Real agents need more than just messages in state. This topic goes deeper into how LangGraph manages state across complex workflows: rich state with multiple fields, state as working memory, and inter-node communication.

LangGraph State Flow: How Data Moves Through the Graph



Key Insight: Each node returns ONLY the fields it changed. LangGraph merges them into the shared state automatically.

Rich State: Multiple Fields for Complex Workflows

State can hold any data your nodes need — counters, flags, collected results, confidence scores. Here is a research agent state with multiple fields:

example_08_langgraph_state_deep_dive.py — Rich State

```
class ResearchState(TypedDict):
    messages: Annotated[list, add_messages]  # Conversation history (reducer: appends)
    topic: str                                # Research topic (plain: replaces)
    facts: List[str]                          # Collected facts (plain: replaces)
    summary: str                              # Final summary
    confidence: float                         # How confident (0.0-1.0)
    iteration: int                            # Which pass are we on
    needs_more_research: bool                 # Should we do another pass?
```

Nodes read what they need and write what they change. The collect_facts node accumulates facts across iterations:

Accumulating Data and Assessing Confidence

```
def collect_facts(state: ResearchState) -> dict:
    iteration = state.get("iteration", 0) + 1
    existing_facts = state.get("facts", [])
```

```
    prompt = f"List 3 key facts about: {state['topic']}"
    if existing_facts:
        prompt += f"\nYou already know: {existing_facts}. Find NEW facts."
    response = llm.invoke(prompt)
    new_facts = [line.strip() for line in response.content.split("\n") if line.strip()]
    return {"facts": existing_facts + new_facts, "iteration": iteration}

def assess_confidence(state: ResearchState) -> dict:
    num_facts = len(state.get("facts", []))
    confidence = min(num_facts / 6.0, 1.0)
    needs_more = confidence < 0.8 and state.get("iteration", 0) < 3
    return {"confidence": confidence, "needs_more_research": needs_more}
```

State as Inter-Node Communication

In LangGraph, nodes communicate ONLY through shared state. There are no function arguments passed between nodes — everything goes through the state dictionary:

Pipeline Pattern — Nodes Communicate Through State

```
class PipelineState(TypedDict):
    raw_input: str
    cleaned: str
    word_list: List[str]
    stats: dict
    final_report: str

def clean_node(state):   # Reads raw_input, writes cleaned
    return {"cleaned": state["raw_input"].strip().lower()}

def split_node(state):   # Reads cleaned, writes word_list
    return {"word_list": state["cleaned"].split()}

def stats_node(state):   # Reads word_list, writes stats
    words = state["word_list"]
    return {"stats": {"word_count": len(words), "unique_words": len(set(words))}}

def report_node(state):  # Reads stats, writes final_report
    s = state["stats"]
    return {"final_report": f"Words: {s['word_count']}, Unique: {s['unique_words']}"}
```

Tip:
Each node reads ONLY what it needs from state and writes ONLY the fields it changes. This keeps nodes decoupled and testable. Use state flags like needs_more_research for routing decisions.

Full source: example_08_langgraph_state_deep_dive.py

Topic 4: LangGraph — Tool Integration

Time: 1 hour

Tool integration in LangGraph requires understanding three components: the @tool decorator for defining tools, bind_tools() for telling the LLM about tools, and ToolNode for executing tool calls. The complete tool-calling loop is: LLM decides to call a tool -> ToolNode executes it -> result goes back to LLM.

The Tool-Calling Loop in Detail

When the LLM's response contains tool_calls, the ToolNode executes each call and returns the results as ToolMessage objects. The agent node then invokes the LLM again with the tool results appended to the messages:

Tool Node with Safety Counter

```
from langgraph.prebuilt import ToolNode

tool_executor = ToolNode(tools)

def tools_node(state: AgentState) -> dict:
    """Execute tool calls and increment the counter."""
    result = tool_executor.invoke(state)
    new_count = state.get("tool_call_count", 0) + len(state["messages"][-1].tool_calls)
    return {"messages": result["messages"], "tool_call_count": new_count}
```

The routing function checks if the LLM wants to call a tool or has finished:

Routing with Safety Valve

```
def should_continue(state: AgentState) -> str:
    last_message = state["messages"][-1]
    if state.get("tool_call_count", 0) >= MAX_TOOL_CALLS:
        print(f"  [Safety] Hit max tool calls ({MAX_TOOL_CALLS}). Forcing end.")
        return "end"
    if hasattr(last_message, "tool_calls") and last_message.tool_calls:
        return "tools"
    return "end"
```

Important

Always add a MAX_TOOL_CALLS safety limit. Without it, a confused LLM could call tools in an infinite loop, wasting API credits and time. 5-10 iterations is usually sufficient for most agent tasks.

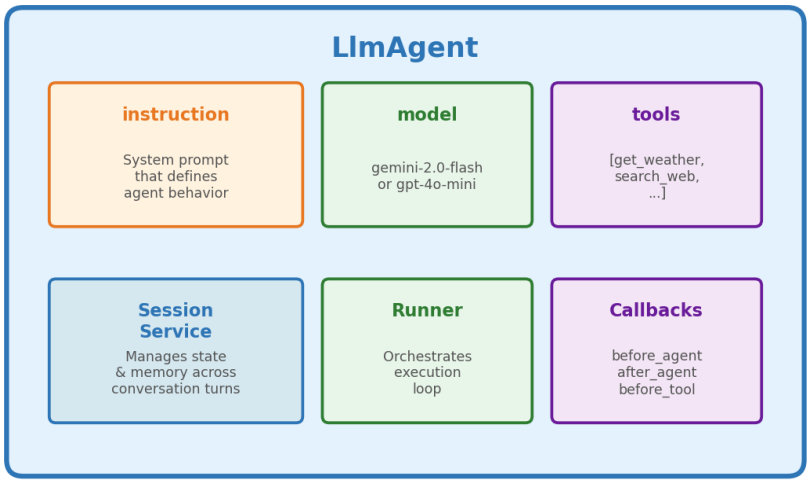
Full source: example_02_langgraph_tool_agent.py

Topic 5: ADK Basics — LlmAgent Structure

Time: 1 hour

Google's Agent Development Kit (ADK) takes a fundamentally different approach from LangGraph. Instead of building a graph manually, you configure an agent declaratively and ADK handles the execution loop for you. The three core components are LlmAgent (the agent), Runner (the execution engine), and InMemorySessionService (the memory store).

Google ADK: LlmAgent Architecture



Creating a Minimal ADK Agent

An LlmAgent needs just three things: name, model, and instruction. The instruction acts as the system prompt:

example_03_adk_agent_basics.py — Minimal Agent

```
from google.adk.agents import LlmAgent
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.genai import types

# Step 1: Create the agent
simple_agent = LlmAgent(
    name="tutor",
    model=os.getenv("GOOGLE_MODEL", "gemini-2.0-flash"),
    instruction="You are a helpful AI tutor. Give clear, concise explanations. "
                "Always end with a follow-up question to check understanding.",
)

# Step 2: Set up Runner and SessionService
session_service = InMemorySessionService()
```

```
runner = Runner(
    agent=simple_agent,
    app_name="tutor_app",
    session_service=session_service,
)
```

Sending Messages and Getting Responses

ADK uses an async event stream. You iterate over events and look for the final response:

Async Event Loop Pattern

```
async def ask_simple(query: str) -> str:
    session = await session_service.create_session(
        app_name="tutor_app", user_id="student",
    )
    response_text = ""
    async for event in runner.run_async(
        user_id="student",
        session_id=session.id,
        new_message=types.Content(
            role="user", parts=[types.Part(text=query)],
        ),
    ):
        if event.is_final_response():
            response_text = event.content.parts[0].text
    return response_text
```

Configuration Options

ADK agents support temperature, max_output_tokens, and description (used in multi-agent systems):

Agent Configuration Options

```
configured_agent = LlmAgent(
    name="creative_writer",
    model="gemini-2.0-flash",
    instruction="You are a creative writing assistant...",
    description="A creative writer that produces vivid, concise prose.",
    generate_content_config=types.GenerateContentConfig(
        temperature=0.9,          # Higher = more creative (0.0-2.0)
        max_output_tokens=200,    # Limit response length
    ),
)
```

Multi-Turn Conversations and Session Isolation

ADK automatically maintains conversation history within a session. Different sessions are completely isolated (like separate chat windows):

Session Memory vs Isolation

```
# Same session = shared memory (agent remembers previous turns)
session = await session_service.create_session(app_name="app", user_id="user1")
# All turns within this session share conversation history
```

```
# Different session = isolated memory (no shared context)
session_a = await session_service.create_session(app_name="app", user_id="user1")
session_b = await session_service.create_session(app_name="app", user_id="user1")
# session_a and session_b have SEPARATE conversation histories
```

Tip:

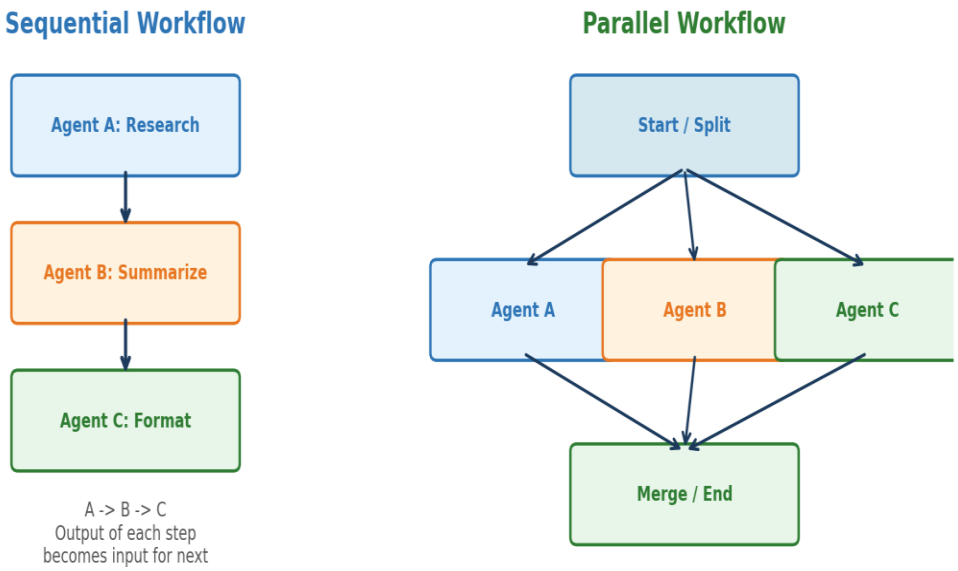
Compare with LangGraph: In LangGraph you manage messages explicitly through state. In ADK, the SessionService handles it automatically. This is the core trade-off: LangGraph gives you full control, ADK gives you convenience.

Full source: example_03_adk_agent_basics.py

Topic 6: ADK — Workflow Agents (Sequential)

Time: 1 hour

ADK supports multi-agent workflows where agents execute in sequence. A SequentialAgent runs a list of sub-agents one after another, passing context between them through the session. This is analogous to a LangGraph linear graph but configured declaratively.



Building a Sequential Pipeline

Three specialized agents run in order: researcher -> analyst -> writer. Each picks up where the previous left off:

example_09_adk_sequential_agent.py — Sequential Pipeline

```
from google.adk.agents import LlmAgent, SequentialAgent

model = os.getenv("GOOGLE_MODEL", "gemini-2.0-flash")

researcher = LlmAgent(
    name="researcher", model=model,
    instruction="You are a research assistant. List 5 key facts about the topic.",
    description="Gathers key facts about a topic.",
)

analyst = LlmAgent(
    name="analyst", model=model,
    instruction="Look at the facts and identify the most important finding, "
                "a surprising detail, and an area for further investigation.",
    description="Analyzes facts and identifies key insights.",
)
```

```
writer = LlmAgent(
    name="writer", model=model,
    instruction="Based on the research and analysis, write a brief summary "
                "paragraph suitable for a newsletter.",
    description="Writes a polished summary from research and analysis.",
)

# SequentialAgent runs them in order
pipeline = SequentialAgent(
    name="research_pipeline",
    sub_agents=[researcher, analyst, writer],
    description="Research -> Analyze -> Write pipeline.",
)
```

Use Cases

Sequential pipelines work well for: research pipelines (gather -> analyze -> summarize), content pipelines (draft -> review -> polish), and data pipelines (extract -> transform -> validate).

Full source: example_09_adk_sequential_agent.py

Topic 7: ADK — AgentTool Delegation

Time: 45 minutes

AgentTool delegation allows a parent agent to use OTHER AGENTS as tools. The parent's LLM decides when to delegate a task to a specialized sub-agent. This is different from SequentialAgent where agents always run in fixed order — with AgentTool, the parent dynamically chooses which specialist to call.

Parent Agent with Specialist Sub-Agents

example_10_adk_agent_delegation.py — AgentTool Pattern

```
from google.adk.tools import AgentTool

math_agent = LlmAgent(
    name="math_expert", model=model,
    instruction="You are a math expert. Use the calculate tool to solve math problems.",
    tools=[calculate],
    description="Solves math problems with step-by-step explanations.",
)

weather_agent = LlmAgent(
    name="weather_expert", model=model,
    instruction="You are a weather expert. Use get_weather to check conditions.",
    tools=[get_weather],
    description="Checks weather conditions for any city worldwide.",
)

# Parent agent wraps specialists as tools
parent_agent = LlmAgent(
    name="coordinator", model=model,
    instruction="You are a coordinator. Delegate math to math_expert "
                "and weather to weather_expert.",
    tools=[
        AgentTool(agent=math_agent),
        AgentTool(agent=weather_agent),
    ],
)
```

Comparing Multi-Agent Patterns

Pattern	How It Works	When to Use
Plain tools	Functions the LLM calls directly	Simple, stateless operations
SequentialAgent	Agents run in fixed order	Pipelines (A then B then C)
AgentTool	Parent chooses which agent to call	Dynamic delegation by task

Full source: example_10_adk_agent_delegation.py

Topic 8: ADK — YAML Configuration

Time: 45 minutes

ADK agents can be configured using YAML files instead of Python code. This separates agent behavior from code, enables quick prompt iteration, and allows non-developers to edit agent instructions.

YAML Configuration Format

agent.yaml Format

```
# agent.yaml -- ADK agent configuration
name: research_assistant
model: gemini-2.0-flash
description: A helpful research assistant that answers questions.

instruction: |
  You are a helpful research assistant.
  Guidelines:
  - Answer questions clearly and concisely
  - If you don't know something, say so
  - Provide sources when possible
  - Keep responses under 200 words

tools:
  - search_web
  - calculate

generate_content_config:
  temperature: 0.7
  max_output_tokens: 500
```

Loading YAML Configuration

Creating Agent from YAML

```
import yaml
config = yaml.safe_load(yaml_config)

agent = LlmAgent(
    name=config["name"],
    model=config.get("model", "gemini-2.0-flash"),
    instruction=config.get("instruction", ""),
    description=config.get("description", ""),
    generate_content_config=types.GenerateContentConfig(
        temperature=config["generate_content_config"].get("temperature", 0.7),
        max_output_tokens=config["generate_content_config"].get("max_output_tokens", 500),
    ),
)
```

When to Use YAML vs Python

YAML Configuration	Python Code
Non-developers edit prompts	Dynamic configuration needed
Quick prompt iteration	Complex tool setup
Config version control	Conditional logic
Standard ADK project layout	Custom agent classes
A/B testing prompts	Integration with other systems

Full source: example_11_adk_yaml_config.py

Topic 9: Comparing LangGraph vs. ADK

Time: 45 minutes

Both frameworks solve the same problem — building AI agents — but with different philosophies. LangGraph gives you explicit control over every aspect of the execution flow. ADK handles the plumbing for you with a declarative configuration. This topic runs the same task in both frameworks to compare developer experience.

LangGraph vs. Google ADK — Feature Comparison

Feature	LangGraph	Google ADK
Approach	Explicit graph (nodes + edges)	Declarative agents (config-based)
State Mgmt	TypedDict with reducers	Session.state dictionary
Tool Binding	.bind_tools() + ToolNode	tools=[...] on LlmAgent
Execution	graph.compile().invoke()	Runner.run_async()
Workflows	Manual graph construction	SequentialAgent / ParallelAgent
Control	High (conditional edges)	Medium (callbacks)
Ecosystem	LangChain / Python	Google Cloud / Gemini
Best For	Complex, custom flows	Quick prototyping, Google stack

Side-by-Side Comparison

Aspect	LangGraph	ADK
Tool definition	@tool decorator	Plain functions
Tool binding	llm.bind_tools()	tools=[...] in agent
Graph building	Manual (StateGraph)	Automatic (hidden)
State management	TypedDict + reducers	Session (automatic)
Routing logic	add_conditional_edges	Built-in loop
LLM providers	Any LangChain LLM	Gemini only
Async required	No (sync by default)	Yes (async only)
Lines of code	~35 lines	~15 lines
Control level	Full (you own the loop)	Limited (ADK owns it)

When to Use Which

Choose LangGraph when you need:

- Full control over the execution flow
- Custom routing logic (multi-agent handoffs, error recovery branches)
- Multiple LLM providers (Groq, OpenAI, Anthropic, etc.)
- Complex state with custom reducers
- Checkpointing and resumable workflows (Week 6)

Choose ADK when you need:

- Rapid prototyping — get an agent running quickly
- Simple tool integration with no boilerplate
- Built-in session management
- Google Cloud deployment (Cloud Run, Vertex AI)
- Multi-agent systems with minimal wiring

Full source: example_05_framework_comparison.py

Topic 10: Basic Middleware — Logging

Time: 45 minutes

Middleware sits between your code and the LLM, intercepting calls to add cross-cutting concerns like logging, timing, and monitoring. In production, you need to know what your agent did, how long it took, and what went wrong.

Structured Logging Setup

Structured Logger Setup

```
import logging, time
from functools import wraps

def setup_logger(name: str = "agent") -> logging.Logger:
    logger = logging.getLogger(name)
    logger.setLevel(logging.INFO)
    if not logger.handlers:
        handler = logging.StreamHandler()
        handler.setFormatter(logging.Formatter(
            "%(asctime)s | %(levelname)-8s | %(name)s | %(message)s",
            datefmt="%H:%M:%S",
        ))
        logger.addHandler(handler)
    return logger
```

Logging Decorator for LangGraph Nodes

The log_node decorator wraps any node function to automatically log start, completion with timing, and errors:

example_12_middleware_logging.py — Logging Decorator

```
def log_node(logger):
    def decorator(func):
        @wraps(func)
        def wrapper(state, *args, **kwargs):
            node_name = func.__name__
            iteration = state.get("iteration", "?")
            logger.info(f"[{node_name}] START | iteration={iteration}")
            start = time.time()
            try:
                result = func(state, *args, **kwargs)
                elapsed = time.time() - start
                logger.info(f"[{node_name}] DONE | {elapsed:.3f}s")
                return result
            except Exception as e:
                elapsed = time.time() - start
                logger.error(f"[{node_name}] FAIL | {elapsed:.3f}s | {type(e).__name__}: {e}")
                raise
        return wrapper
    return decorator

# Usage:
@log_node(logger)
```

```
def agent_node(state: AgentState) -> dict:
    response = llm_with_tools.invoke(state["messages"])
    return {"messages": [response], "iteration": state.get("iteration", 0) + 1}
```

ADK Logging via Event Inspection

In ADK, add logging by inspecting all events in the async loop:

ADK Event Logging

```
async for event in runner.run_async(user_id="user1", session_id=session.id, new_message=msg):
    if event.content and event.content.parts:
        for part in event.content.parts:
            if hasattr(part, "function_call") and part.function_call:
                logger.info(f"[event] tool_call: {part.function_call.name}")
            elif hasattr(part, "text") and part.text and event.is_final_response():
                logger.info(f"[event] final_response: {part.text[:80]}")
```

What to Log vs What Not to Log

Log This	Do NOT Log This
Every LLM call (start, end, duration)	Full prompt text in production
Every tool call (name, args, result)	Full LLM responses (use trace IDs)
Routing decisions	Sensitive user data (PII, credentials)
Errors and retries	Internal state dumps
Total query duration	Authentication tokens

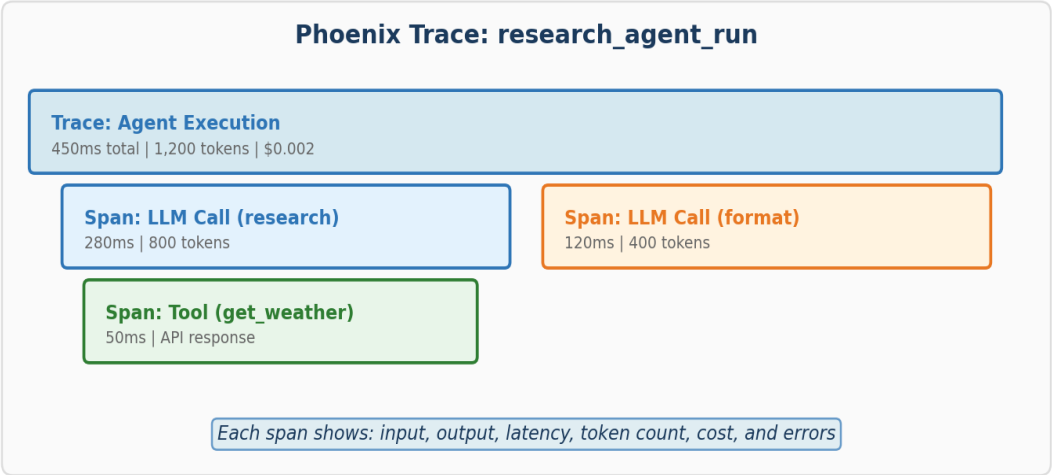
Full source: example_12_middleware_logging.py

Topic 11: Tracing Simple Flows in Phoenix

Time: 1 hour

Arize Phoenix provides a visual dashboard to inspect every LLM call, tool call, and agent loop iteration. It uses OpenTelemetry for instrumentation and gives you spans, traces, and metrics for understanding agent behavior.

Arize Phoenix: Tracing Agent Executions



Setting Up Phoenix Tracing

example_13_phoenix_tracing.py — Phoenix Setup

```
import phoenix as px
from openinference.instrumentation.langchain import LangChainInstrumentor
from phoenix.otel import register

px.launch_app()                          # Start the dashboard
tracer_provider = register()             # Set up OpenTelemetry
LangChainInstrumentor().instrument(tracer_provider=tracer_provider)

# Dashboard: http://localhost:6006
# Now ALL LangChain/LangGraph calls are automatically traced!
```

What to Look For in Phoenix

- **Traces Tab:** Each query appears as a separate trace. Click to see the full span tree.
- **Span Tree:** Root span = full agent invocation. Child spans = individual LLM calls. Grandchild spans = tool executions.

- **Span Details:** Input (prompt), Output (response), Metadata (model, token count, latency), Tool calls (function name, arguments).
- **Key Metrics:** Latency per call, Token count (input vs output), Error rate.

A simple question (no tools) creates ~1 span. A tool-using query creates ~3 spans (LLM -> tool -> LLM). A multi-tool query creates 5+ spans. Phoenix visualizes this as a tree so you can understand exactly what your agent did.

Tip:
Install Phoenix: pip install arize-phoenix openinference-instrumentation-langchain. The dashboard runs locally at http://localhost:6006 and requires no account or API key.

Full source: example_13_phoenix_tracing.py

Topic 12: Why Learn Frameworks Before Patterns?

Time: 45 minutes

The curriculum teaches frameworks (LangGraph, ADK) in Week 2 before jumping into advanced patterns (ReAct, multi-agent, etc.) in Weeks 3-4. Understanding this order helps you learn more effectively.

The Problem with Raw LLM Calls

In Week 1, you made direct LLM calls: response = llm.invoke("What is AI?"). This works for simple questions, but real agents need tool calling, loops, state management, error recovery, and observability. Building all of this from scratch requires 100+ lines of boilerplate that is the same for every agent.

What Frameworks Give You

LangGraph provides:

- StateGraph — visual, explicit control flow
- ToolNode — automatic tool execution
- add_messages — conversation history management
- Conditional edges — routing logic
- Checkpointing — save/resume agent state

ADK provides:

- LlmAgent — declarative agent configuration
- Automatic tool loop — no manual wiring
- Session management — built-in memory
- SequentialAgent, AgentTool — multi-agent patterns
- Google Cloud deployment — production-ready

The Key Insight

Remember

PATTERNS are WHAT your agent does (strategy). FRAMEWORKS are HOW you build it (implementation). You need the HOW before you can build the WHAT. The ReAct pattern (Week 3) is just the agent -> tools -> agent loop you already built in Week 2!

Full source: example_14_why_frameworks.py

Topic 13: Hands-on — Port Agent Between Frameworks

Time: 1 hour

This topic walks through implementing the same task in both LangGraph and ADK. The key insight is that tool logic should be framework-agnostic — write it once, wrap it differently per framework.

Shared Tool Logic

example_05_framework_comparison.py — Shared Logic

```
# Write tool logic ONCE as plain functions
def _calculate_logic(expression: str) -> str:
    try:
        allowed = set("0123456789+-*/.() ")
        if not all(c in allowed for c in expression):
            return "Error: Invalid characters"
        return f"{expression} = {eval(expression)}"
    except Exception as e:
        return f"Error: {e}"

def _reverse_logic(text: str) -> str:
    return text[::-1]
```

LangGraph Wrapper

LangGraph Version

```
# LangGraph: wrap with @tool decorator
@tool
def calculate(expression: str) -> str:
    """Evaluate a math expression."""
    return _calculate_logic(expression)

# Build the full graph (StateGraph, nodes, edges)
graph = StateGraph(AgentState)
graph.add_node("agent", agent_node)
graph.add_node("tools", ToolNode(tools))
# ... ~35 lines total
```

ADK Wrapper

ADK Version

```
# ADK: pass plain functions directly (no decorator!)
def calculate(expression: str) -> str:
    """Evaluate a math expression."""
    return _calculate_logic(expression)

agent = LlmAgent(
    name="comparison_agent", model="gemini-2.0-flash",
    instruction="Use calculate and string_reverse tools.",
    tools=[calculate, string_reverse],
)
```

```
# ... ~15 lines total
```

Tip:
This exercise (Exercise 3) asks you to build a framework switcher that runs the same query on both frameworks and compares results. It teaches you to think about tools as framework-agnostic business logic.

Full source: example_05_framework_comparison.py

Topic 14: Common Setup Pitfalls

Time: 45 minutes

These are the most common issues students encounter when setting up LangGraph and ADK. Knowing these saves hours of debugging.

Pitfall 1: Missing .env Configuration

Symptom: AuthenticationError: Invalid API key. Fix: Copy config/.env.example to config/.env and add your actual keys WITHOUT quotes:

.env Configuration

```
# config/.env (no quotes around values!)
GROQ_API_KEY=gsk_abc123...
GOOGLE_API_KEY=AIza...
OPENAI_API_KEY=sk-...
LLM_PROVIDER=groq
```

Pitfall 2: Import Errors

Common package name mismatches:

Import Name	pip Package
langchain_groq	langchain-groq
langchain_openai	langchain-openai
google.adk	google-adk
google.genai	google-genai
phoenix	arize-phoenix

Pitfall 3: Groq/Llama Malformed Tool Calls

Groq's Llama models sometimes generate invalid tool call JSON. Always wrap LLM calls in retry logic (3 retries is standard).

Pitfall 4: ADK Requires async/await

ADK's runner.run_async() is an async generator. You MUST use 'async for', not a regular for loop. Use asyncio.run() to call async functions from synchronous code.

Pitfall 5: State Management Mistakes

- Forgetting add_messages reducer — messages get REPLACED instead of APPENDED
- Not providing all initial state fields — causes KeyError
- Returning full state from nodes instead of just changed fields

Pitfall 6: Framework-Provider Mismatch

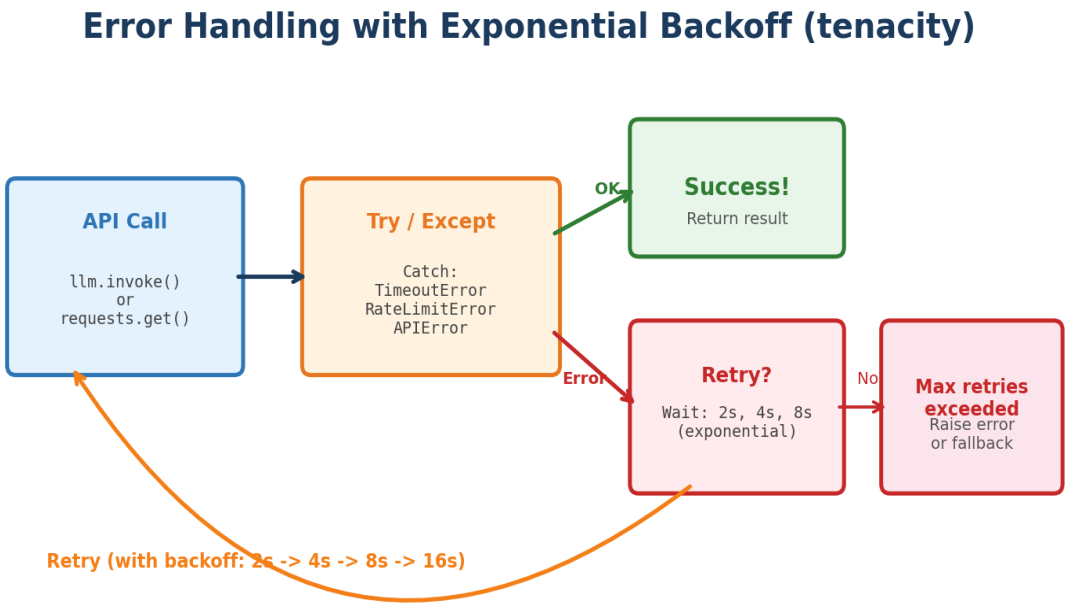
ADK ONLY works with Google Gemini models. LangGraph works with any LangChain-supported LLM (Groq, OpenAI, Anthropic, Google, etc.).

Full source: example_15_setup_pitfalls.py

Topic 15: Error Handling Fundamentals

Time: 1 hour

Real-world agents face API timeouts, rate limits, malformed tool calls, and tool execution failures. This topic teaches systematic error handling patterns for both LangGraph and ADK.



Pattern 1: Retry with Exponential Backoff

example_06_error_handling.py — Retry Pattern

```
def retry_with_backoff(func, max_retries=3, base_delay=1.0):
    """Retry a function with exponential backoff.
    Delays: 1s, 2s, 4s, 8s, ..."""
    for attempt in range(max_retries):
        try:
            return func()
        except Exception as e:
            if attempt == max_retries - 1:
                raise  # Last attempt - re-raise
            delay = base_delay * (2 ** attempt)
            print(f"  [Retry {attempt + 1}/{max_retries}] {type(e).__name__}: {e}")
            print(f"  [Waiting {delay:.1f}s before retry...]")
            time.sleep(delay)
```

Pattern 2: Safe Tool Error Handling

Tools should return error MESSAGES instead of raising exceptions. This lets the LLM read the error and try a different approach:

Safe Tool Pattern

```
# BAD: crashes on bad input
def bad_divide(a: float, b: float) -> float:
    return a / b  # Crashes on b=0!

# GOOD: returns error message
def safe_divide(a: float, b: float) -> str:
    try:
        if b == 0:
            return f"Error: Cannot divide {a} by zero."
        return f"{a} / {b} = {a/b:.4f}"
    except Exception as e:
        return f"Error: Could not compute {a}/{b}: {e}"
```

Pattern 3: LangGraph Error-Aware State

Error-Aware State and Routing

```
class ErrorAwareState(TypedDict):
    messages: Annotated[list, add_messages]
    error_count: int       # How many errors encountered
    iteration_count: int   # Total loop iterations

MAX_ITERATIONS = 6   # Prevent infinite loops
MAX_ERRORS = 3        # Give up after too many errors

def should_continue(state: ErrorAwareState) -> str:
    if state.get("iteration_count", 0) >= MAX_ITERATIONS:
        return "end"   # Max iterations reached
    if state.get("error_count", 0) >= MAX_ERRORS:
        return "end"   # Too many errors
    last_msg = state["messages"][-1]
    if hasattr(last_msg, "tool_calls") and last_msg.tool_calls:
        return "tools"
    return "end"
```

Error Handling Best Practices

- TOOLS: Return error messages, never raise exceptions
- LLM CALLS: Retry with exponential backoff (1s, 2s, 4s), max 3 retries
- LOOPS: Always set a maximum iteration count (5-10 is usually enough)
- GRACEFUL DEGRADATION: If everything fails, return a user-friendly fallback message
- ADK-SPECIFIC: Use error-aware instructions to tell the agent how to handle tool failures
- LANGGRAPH-SPECIFIC: Track error_count in state and route to fallback nodes

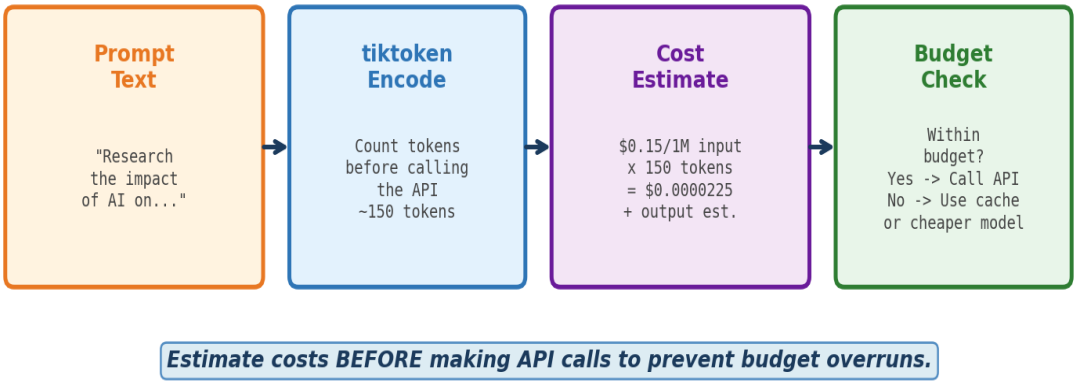
Full source: example_06_error_handling.py

Topic 16: Cost-Aware API Calls

Time: 45 minutes

LLM API calls cost money. Multi-tool agents make MULTIPLE LLM calls per query (one per loop iteration), so costs multiply quickly. A 3-tool query costs 3x a simple question. This topic shows how to track token usage, estimate costs, and set budget alerts.

Cost-Aware API Calls: Token Estimation Pipeline



Extracting Token Usage from LLM Responses

example_07_cost_tracking.py — Token Usage

```
response = llm.invoke([SystemMessage(content="..."), HumanMessage(content="...")])

# Token usage is in response_metadata (format varies by provider)
metadata = response.response_metadata
usage = metadata.get("token_usage", metadata.get("usage", {}))
input_tokens = usage.get("prompt_tokens", usage.get("input_tokens", 0))
output_tokens = usage.get("completion_tokens", usage.get("output_tokens", 0))
total_tokens = usage.get("total_tokens", input_tokens + output_tokens)
```

Using the CostTracker Utility

Cost Tracking in Agent Nodes

```
from shared.utils.cost_tracker import CostTracker

tracker = CostTracker(weekly_budget=1.00)

# Log each API call
```

```
cost = tracker.log_call("llama-3.3-70b-versatile", input_tokens=150, output_tokens=300)

# Inside a LangGraph agent node:
def tracked_agent_node(state):
    response = llm_with_tools.invoke(state["messages"])
    usage = response.response_metadata.get("token_usage", {})
    cost = tracker.log_call(model_name,
        usage.get("prompt_tokens", 0),
        usage.get("completion_tokens", 0))
    print(f"  [cost] $\{cost:.6f}")
    return {"messages": [response]}

tracker.report()  # Print cumulative cost summary
```

Model Cost Comparison

Model	Cost Tier	Notes
Gemini Flash	Free	Great for learning and prototyping
Groq (Llama 3.3)	Very Low (~$0.001/query)	Affordable for development
GPT-4o-mini	Low (~$0.01/query)	Good quality/price balance
GPT-4o	High (~$0.05/query)	10-50x more expensive than Groq
Claude Sonnet	Medium	Strong reasoning capabilities

Cost Awareness

Tool agents multiply costs: each loop iteration is another LLM call. A 3-tool query costs 3x a simple question. Set weekly budgets with CostTracker to avoid bill shock. Use Gemini Flash (free) or Groq (very low cost) for learning.

Full source: example_07_cost_tracking.py

Hands-On Exercises

Complete all three exercises to solidify your understanding. Each exercise has scaffolded code with TODO comments. Look at the referenced example files for guidance.

Exercise 1: Calculator Agent (LangGraph)

Difficulty: Beginner | Time: 1.5 hours

File: exercises/exercise_01_calculator_agent.py

Build a LangGraph agent with calculator tools that can solve multi-step math problems. The agent should chain tool calls (e.g., 'add 15 and 27, then multiply by 3').

What You Need to Do:

- 1. Complete the 3 tool functions: add, multiply, divide
- 2. Handle division by zero in the divide tool (return error message, don't crash!)
- 3. Set up the LLM with provider flexibility (groq/openai)
- 4. Build the StateGraph with agent_node and ToolNode
- 5. Add conditional edges for the agent -> tools -> agent loop
- 6. Test with all 3 queries: simple addition, multi-step, division by zero

Here is the scaffolded tool code to complete:

Exercise 1 — TODO Scaffolding

```
@tool
def add(a: float, b: float) -> str:
    """Add two numbers together."""
    # TODO: Return a string like "15.0 + 27.0 = 42.0"
    pass

@tool
def divide(a: float, b: float) -> str:
    """Divide a by b. Handles division by zero gracefully."""
    # TODO: Check if b is zero - return error message string
    # Otherwise return the result as a string
    pass
```

Tip:
Look at example_02_langgraph_tool_agent.py for the full pattern. Tools should return STRINGS (not numbers) because the LLM reads them. Division by zero should return an error message, NOT crash.

Exercise 2: Multi-Tool ADK Agent

Difficulty: Beginner-Intermediate | Time: 2 hours

File: exercises/exercise_02_multi_tool_adk_agent.py

Build an ADK agent with 3 tools that can handle multi-step queries like 'Convert 100°F to Celsius, then round to nearest integer'.

What You Need to Do:

- 1. Complete the 3 tool functions: calculate, to_uppercase, convert_temperature
- 2. Create an LlmAgent with clear instructions mentioning each tool
- 3. Set up Runner and InMemorySessionService
- 4. Create the ask_agent helper function (async)
- 5. Test with all 3 queries
- 6. Bonus: Show session persistence across turns

Tip:
ADK tools are plain functions — NO @tool decorator needed. ADK reads your type hints and docstrings to create tool schemas. Look at example_04_adk_tool_agent.py for the pattern.

Exercise 3: Framework Switcher

Difficulty: Intermediate | Time: 2.5 hours

File: exercises/exercise_03_framework_switcher.py

Build an agent system where the SAME tools and SAME query can run on either LangGraph or ADK, switchable via a parameter. Then compare the outputs from both frameworks.

What You Need to Do:

- 1. Complete the shared tool functions (plain Python — no decorators)
- 2. Implement build_langgraph_agent() that wraps tools for LangGraph
- 3. Implement build_adk_agent() that uses tools with ADK
- 4. Implement run_with_framework() that dispatches to the right framework
- 5. Implement compare_frameworks() that runs both and prints results
- 6. Test with the provided queries

Tip:
Write tool logic ONCE as plain functions, then wrap differently per framework. For LangGraph use @tool decorator when wrapping. For ADK pass plain functions directly. Look at example_05_framework_comparison.py for the pattern.

Week 2 Ongoing Research Project

Your ongoing project across all 9 weeks is building a Topic Research Agent. Each week adds new capabilities. By Week 2's end, your agent should be framework-based with error handling, cost tracking, and observability.

Week 2 Deliverable

"Framework-based basic agent with error handling and cost awareness."

Project Milestones

Milestone 1: LangGraph Basics (2 hours)

Create a simple graph for summary generation. Your research agent should use a StateGraph with at least 3 nodes:

- research_node: Uses an LLM to gather facts about a topic
- format_node: Structures the raw research into a clean output
- A routing function that decides if more research is needed

Reference: project-topic-research-agent/src/agents/langgraph_agent.py

Milestone 2: ADK Basics (2 hours)

Use LlmAgent for the same research task. Build an ADK version of your research agent:

- Create an LlmAgent with research-focused instructions
- Set up Runner and InMemorySessionService
- Test with the same topics as your LangGraph version

Reference: project-topic-research-agent/src/agents/adk_agent.py

Milestone 3: Error Handling (1 hour)

Wrap API calls with try/except and exponential backoff using the tenacity library or manual retry logic. Handle these specific scenarios:

- RateLimitError — retry with exponential backoff
- Timeout — retry with shorter query or fallback response
- Malformed responses — retry up to 3 times (common with Groq/Llama)
- Tool execution failures — return error messages, not exceptions

Milestone 4: Middleware & Cost Tracking (1 hour)

Add logging middleware and build a token counter:

- Add the log_node decorator to all LangGraph agent nodes
- Implement CostTracker to estimate cost per API call using tiktoken
- Set a weekly budget and log warnings when approaching the limit
- Print a cost report after each agent run

Milestone 5: Trace with Phoenix (1 hour)

Use Arize Phoenix to visualize your agent's execution:

- Install and configure Phoenix with LangChainInstrumentor
- Run your research agent and inspect the trace in the Phoenix dashboard
- Identify how many LLM calls your agent makes per query
- Commit your Week 2 code to the repository

Practice Problems

LangGraph Basics (3 hours):

"Build a simple graph agent for greeting; add state." Create a graph that takes a user's name, generates a personalized greeting, and tracks conversation turns in state.

ADK Basics (3 hours):

"Create LlmAgent for summary; use YAML." Build an ADK summary agent, then refactor its configuration into a YAML file.

Error Handling Lab (2 hours):

"Deliberately introduce failures (invalid API key, network timeout simulation) and build an agent that handles each gracefully with retry logic and fallback responses."

Summary & Key Takeaways

- LangGraph gives you explicit control: StateGraph, nodes, edges, conditional routing, and state reducers
- ADK gives you declarative convenience: LlmAgent, Runner, SessionService, SequentialAgent, AgentTool
- The ReAct loop (agent -> tools -> agent) is the core pattern for tool-using agents in both frameworks
- State management is the foundation: use TypedDict + add_messages in LangGraph, sessions in ADK
- Always handle errors gracefully: retry with backoff, return error messages from tools, set iteration limits
- Track costs: every LLM call has input + output tokens, tool agents multiply costs per loop iteration
- Add logging middleware to every production agent: structured format, timing, error tracking
- Use Phoenix for visual tracing: see exactly what your agent did, how long it took, and what went wrong
- Write tool logic as framework-agnostic functions, then wrap per framework

What's Next: Week 3 — Basic Agentic Patterns

In Week 3, you will learn the core agentic patterns that are IMPLEMENTED using the frameworks from Week 2:

- Reflection pattern — agents that critique and refine their own output
- Chain-of-Thought — step-by-step reasoning for complex problems
- Human-in-the-Loop — agents that ask for user confirmation before acting
- Tool Use patterns — what makes a good tool, tool schemas, and selection strategies

Everything from Week 3 onward builds on the framework foundations you mastered this week.

Resource Links

- LangGraph Official API: https://langchain-ai.github.io/langgraph/reference/graphs/
- ADK Documentation: https://google.github.io/adk-docs/
- Phoenix User Guide: https://docs.arize.com/phoenix/user-guide
- Week 2 Repository: https://github.com/kikiaitraining/ai-agent-mastery/tree/master/week-02-framework-basics